

VULNERABILITY ASSESSMENT REPORT

OWASP Juice Shop Security Assessment

Internship Program: Future Interns

Track: Cyber Security

Task: Task 1 – Vulnerability Assessment

Repository: FUTURE_CS_01

Prepared By: _____IGNEVOURYA EHNO HERMANN_____

Date: _____12/05/2026_____

Table of Contents

INTRODUCTION.....	3
Objectives	3
I. Tools used.....	3
II. Methodology	3
II. Active Scanner.....	4
1. Identified risk	8
2. Safety impact.....	8
3. Recommendation.....	8
4. Risk level	9
5. Safety impact.....	9
6. Recommendation.....	9
IV. Recommendations.....	10
CONCLUSION	11

LIST OF FIGURES

Figure 1: General interface.....	4
Figure 2: Nmap result.....	5
Figure 3 : OWASP ZAP alerts	6
Figure 4 : Browser DevTools analysis.....	7
Figure 5 : Browser DevTools analysis.....	8

INTRODUCTION

This report presents the results of a vulnerability assessment conducted on the OWASP Juice Shop web application using professional cybersecurity tools.

The purpose of this assessment was to identify common vulnerabilities, rank risks, and provide recommendations for remediation.

Objectives

- Identify common web vulnerabilities
- Perform passive vulnerability scanning
- Classify security risks
- Provide remediation recommendations
- Document findings professionally

I. Tools used

Tool	Objective
Kali Linux	Security Test Environment
Nmap	Network and Service Discovery
OWASP ZAP	Passive vulnerability analysis
Browser Development Tools	Application and cookie inspection
	Professional report design

II. Methodology

1. Reconnaissance using Nmap
2. Passive Scanning Using OWASP ZAP
3. Browser-based scanning using DevTools

4. Risk classification
5. Safety recommendations
6. Summary of Findings

Vulnerability	Risk level	Impact	Recommendation
Missing Security Headers	Low	Weak browser protection	Implement CSP and HSTS
XSS potential	Medium	Session hijacking possible	Validate User Input
Disclosure of Information	Medium	Exposure to sensitive data	Reduce unnecessary disclosures

II. Active Scanner

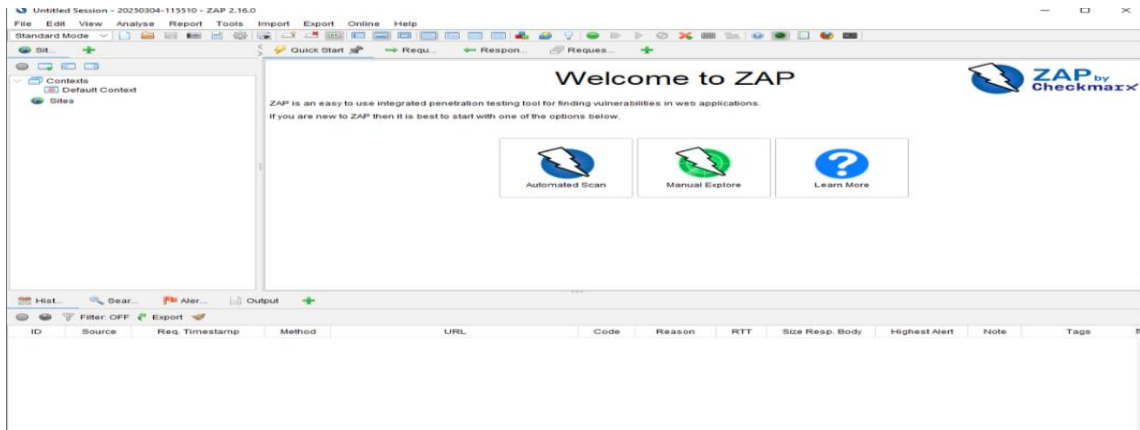


Figure 1 : General interface

Before we explore the interface, let's start by capturing web traffic. This will be more meaningful once we have data to analyze.

ZAP offers two modes of operation:

Automatic scanning: It crawls the site and tests its vulnerabilities from a simple URL. This approach is fast but lacks precision and control. It is therefore not recommended.

Manual Crawl: ZAP acts as a proxy between the browser and the server. The auditor can observe and manipulate the exchanges, then run targeted tests with the built-in tools.

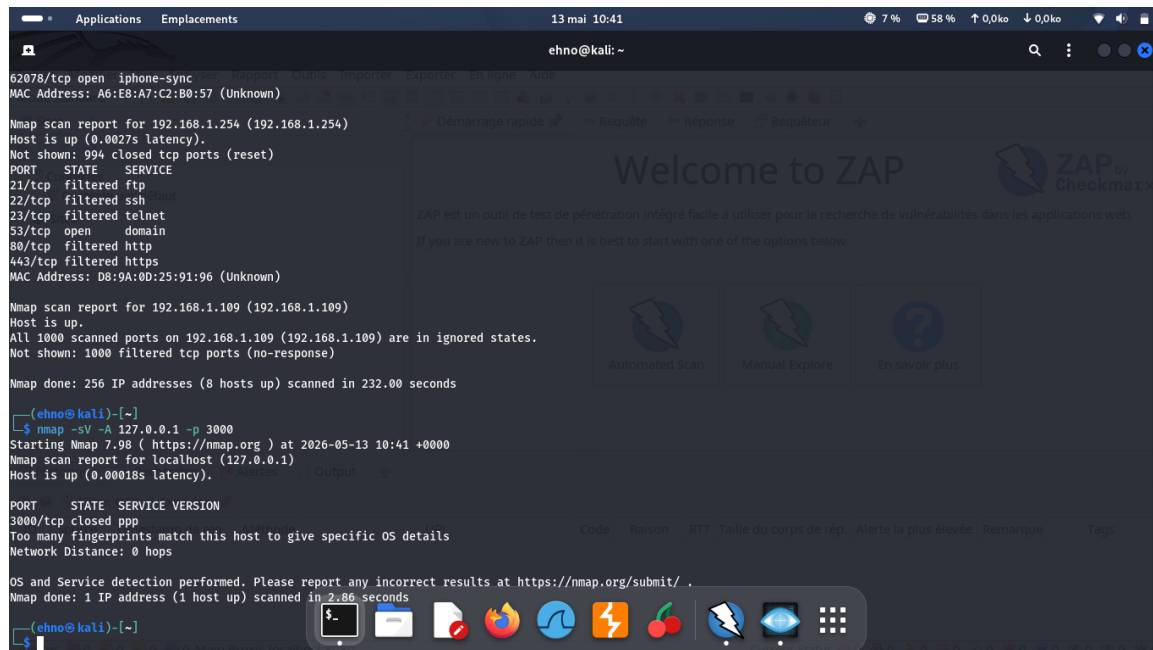


Figure 2 : Nmap result

We ran the site scan to find any open port to launch our attack on the site.

Here we notice three box identify:

1. The box at which we put the link of the site to attack.
2. Secondly, we can discover all the information on our site, because it is vulnerable and we have had access.
3. The alerts discovered during the scan of this site <http://localhost:3000> : high, medium and proxy.

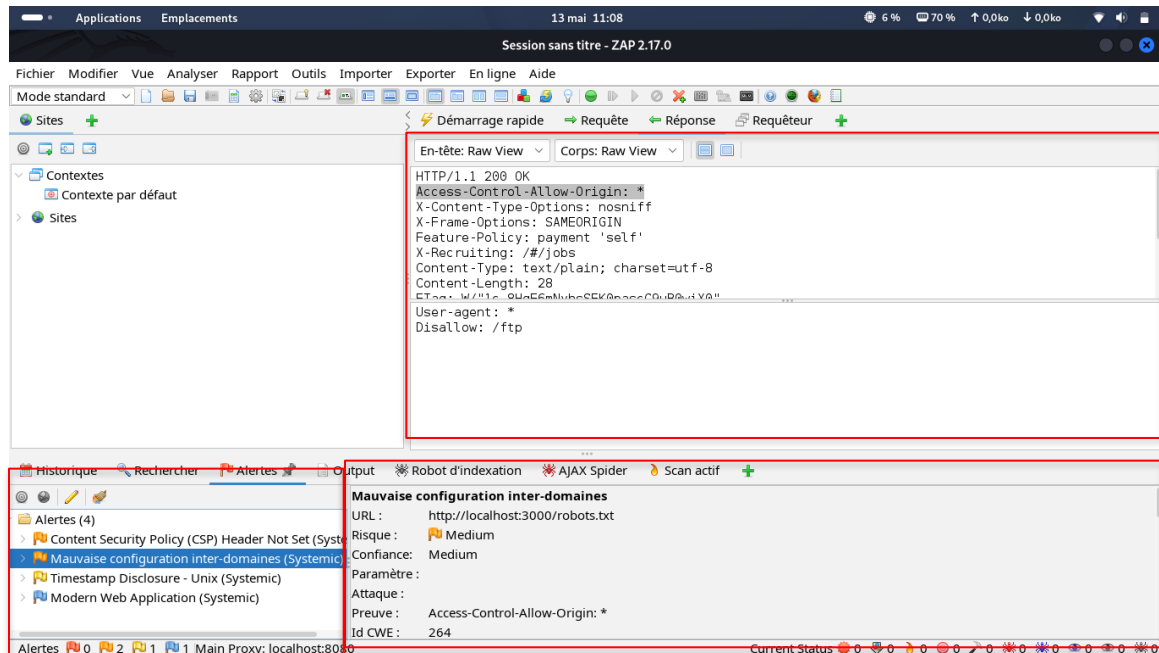


Figure 4 : Browser DevTools analysis

This image shows the alerts detected by OWASP ZAP during passive application scanning.

Vulnerabilities displayed include:

- Content Security Policy (CSP) Header Not Set
- Cross-domain misconfiguration
- Timestamp Disclosure
- Modern Web Application

Active alert is about: Cross-domain misconfiguration

1. Identified risk

Server Uses: Access-Control-Allow-Origin

This means that:

- All external origins are allowed
- CORS control is too permissive

2. Safety impact

This misconfiguration can:

- Allow unauthorized requests
- Expose sensitive data
- Facilitate certain web attacks

3. Recommendation

Limit Allowed Domains in: Access-Control-Allow-Origin

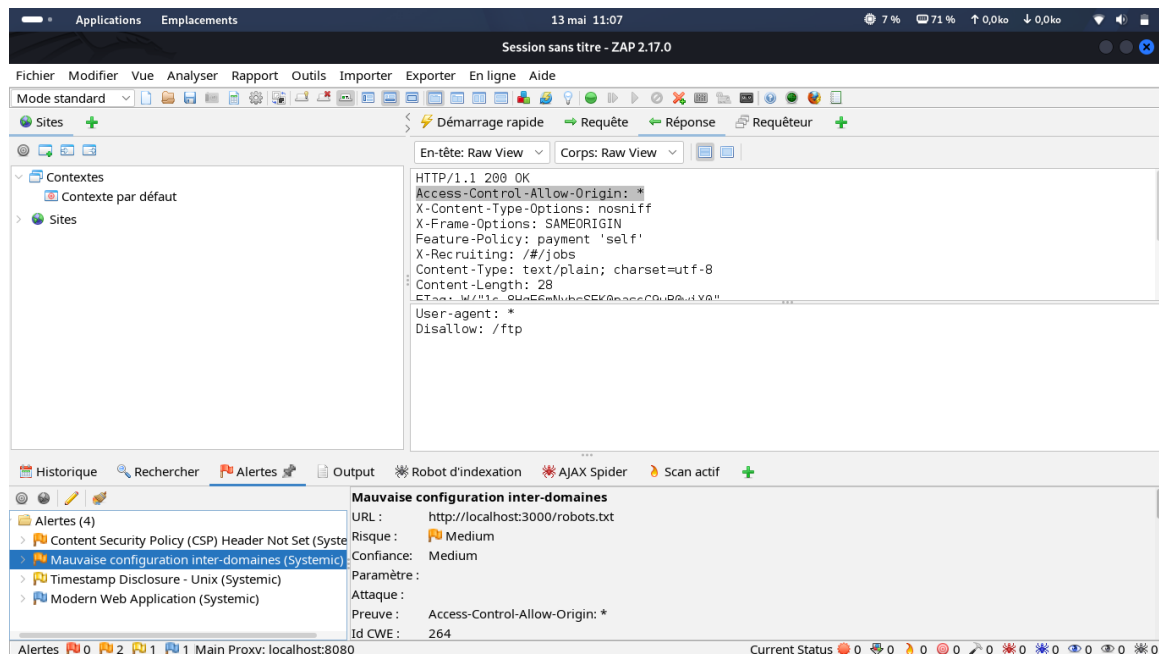


Figure 5 : Browser DevTools analysis

This screenshot shows a vulnerability of type: Timestamp Disclosure – Unix, detected by OWASP ZAP.

The app discloses:

- Unix system date/time information
- internal server metadata

4. Risk level

Risk is considered: Low / Medium

5. Safety impact

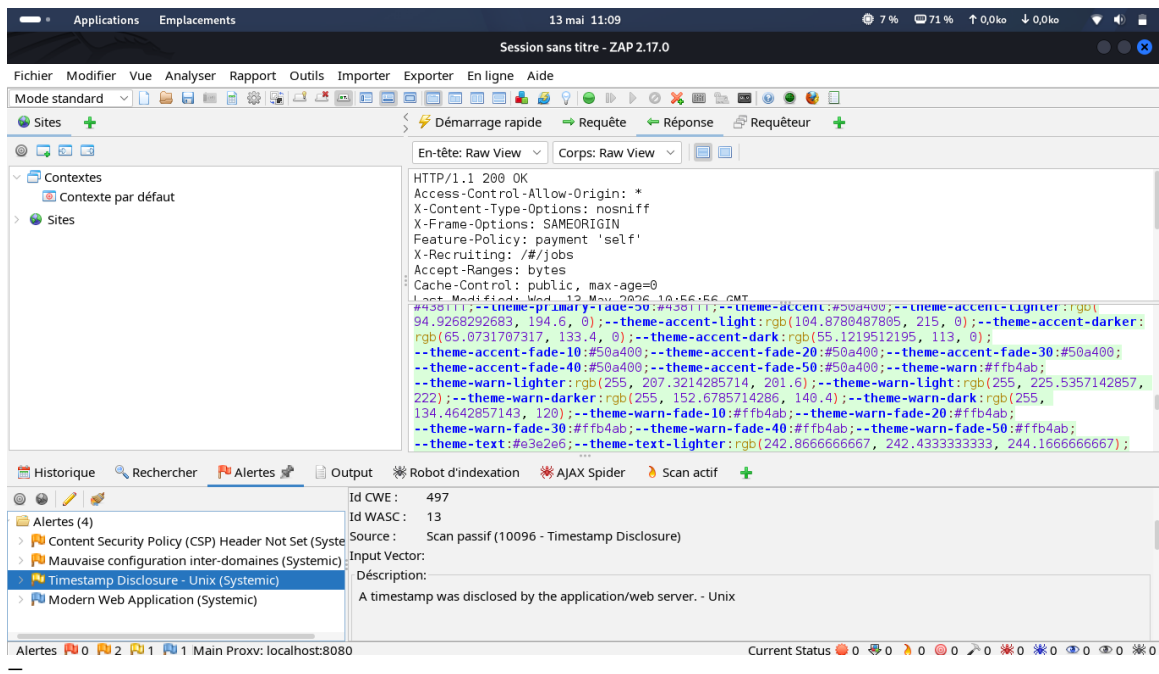
Although this vulnerability is not critical, it can:

- Helping an attacker gather information
- Facilitate system recognition
- Improve targeted attacks

6. Recommendation

Reduce the information exposed in:

- headers HTTP
- Server Responses
- System metadata



IV. Recommendations

- Implement secure HTTP headers
- Enable HTTPS and HSTS
- Validate and sanitize user input
- Secure cookies using HttpOnly and Secure flags
- Conduct regular security assessments

CONCLUSION

The vulnerability assessment identified several low- and medium-risk security issues in the OWASP Juice Shop application.

Improving security headers, input validation, and cookie protection would significantly strengthen the overall security posture of the application.